

Capturing end-to-end provenance for machine learning pipelines

Marius Schlegel^{ID*}, Kai-Uwe Sattler^{ID}

TU Ilmenau, Germany

ARTICLE INFO

Dataset link: <https://github.com/mariusschlegel/mlflow2prov/>

Keywords:

Machine learning model development
Data science
ML pipelines
Artifact management
Provenance
MLflow
W3C PROV

ABSTRACT

Modern workflows for developing ML pipelines utilize ML artifact management systems (ML AMSs) such as MLflow in addition to traditional version control systems such as Git. ML AMSs collect data, model, metadata and software artifacts used and produced in pipeline development workflows. While ensuring repeatability and reproducibility, the provenance capabilities are still rudimentary, mainly due to incomplete traces, coarse granularity, and limited query capabilities. In this paper, we introduce a comprehensive PROV-compliant provenance model that captures end-to-end provenance traces of ML pipelines, their artifacts, and their relationships based on MLflow and Git activities. Moreover, we present the tool MLflow2PROV for continuously extracting provenance graphs according to our model, enabling querying, analyzing, and processing of the collected provenance information.

1. Introduction

The development of machine learning (ML) models typically requires highly exploratory and iterative approaches [1,2]: From data ingestion, data processing and feature engineering, to model design and algorithm selection, to model training and optimization – it is often a long journey to produce a model that maximizes target metrics. Each step, iteration, and evolution of a pipeline produces artifacts, such as data, model, metadata and software artifacts. As a result, dealing with reproducibility and traceability of ML pipeline artifacts is often quite challenging [3].

In addition to version control systems (VCSs), ML artifact management systems (ML AMSs) [4–8] are core tools for ML developers and data scientists to systematically collect ML pipelines, experiments, and their artifacts, and to allow repeating experiment runs and reproducing artifacts and results. A popular open-source representative is MLflow [9,10]. Its experiment tracking component captures data, model, metadata and software artifacts based on log statements, with automatic logging significantly reducing manual effort and intrusiveness. Reusability, repeatability and reproducibility are achieved by individual model and project packaging formats.

Despite their growing adoption, ML AMSs, including MLflow, still have only rudimentary provenance capabilities [3]: First, the focus of the provided provenance information is mainly on model training and model evaluation. For example, data preprocessing and feature transformation steps are often not reflected in terms of provenance. Thus, a holistic view on end-to-end pipeline execution traces cannot

be represented due to missing relationships and coarse granularity of entities. Second, the development history of a pipeline or an experiment is typically not reflected. However, this is particularly relevant for collaborative environments to identify which changes, when, and by whom have led to an improvement or degradation of metrics. Third, searching for provenance information is very limited and only possible via UI, Python, or REST API. Thus, querying a holistic provenance data model using relational or graph query languages is not possible. Fourth, ad-hoc provenance data models lack well-known semantics such as W3C PROV [11]. Thus, the provenance documentation cannot be exported as interoperable formats, which makes it difficult to use in distributed use cases, e.g., federated learning [12]. In addition, the provenance information cannot be easily consumed by other tools for further processing, analysis, and visualization [13–15].

Several approaches have attempted to address these gaps. Ontology-based solutions, such as MEX [16], ML Schema [17], and PROV-ML [18], define vocabularies to represent ML workflows but often lack integration with practical toolchains like MLflow and Git. Tools like DLProv [19] and Geyser [20] provide provenance support but primarily target specific stages of the ML pipeline, such as model training, offering only a partial view of end-to-end workflows. Additionally, these solutions rarely consider VCSs, which are pivotal in capturing the iterative and collaborative nature of ML development. This fragmented coverage leaves a significant gap in achieving comprehensive and actionable provenance in ML projects.

To address these issues, we argue to build upon existing toolchains used by ML developers and data scientists and, thus, to transparently

* Corresponding author.

E-mail addresses: marius.schlegel@tu-ilmenau.de (M. Schlegel), kus@tu-ilmenau.de (K.-U. Sattler).

¹ <https://github.com/mariusschlegel/mlflow2prov/>

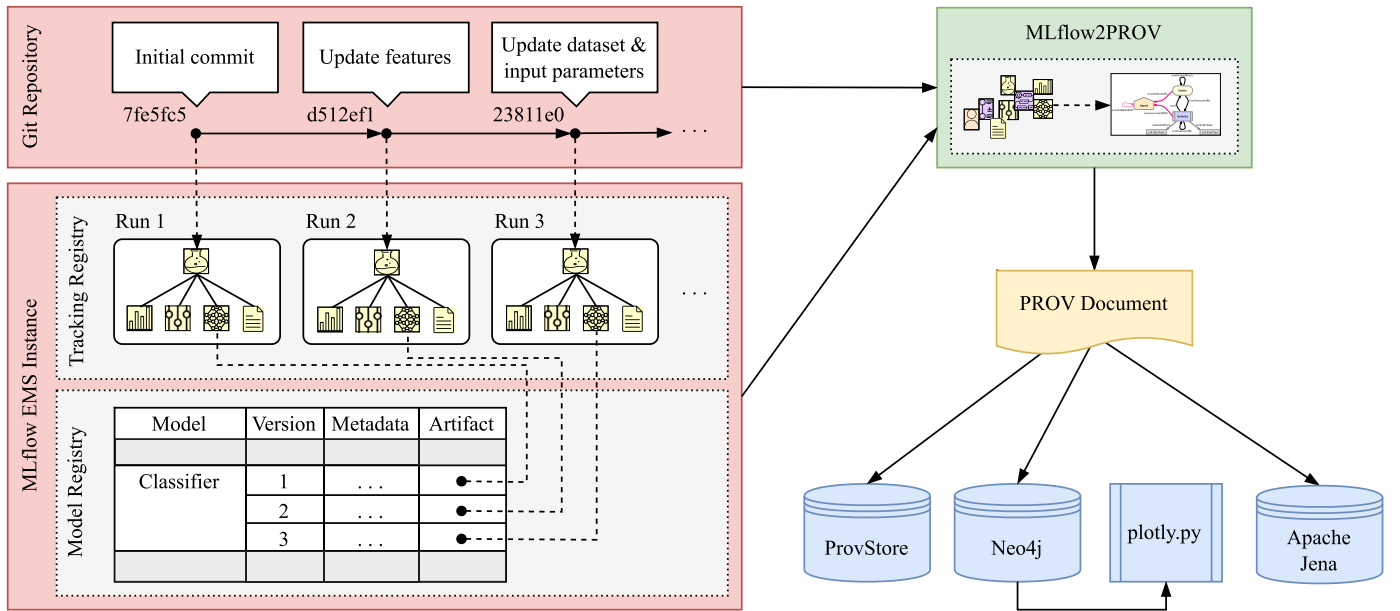


Fig. 1. The evolution and execution of an ML experiment pipeline utilizing the Git VCS and the MLflow AMS as input for the provenance extraction and provenance graph generation with MLflow2PROV.

integrate provenance support. Following our preliminary work [21, 22], our approach to minimally invasive provenance support is as follows: Based on provenance-relevant metadata extracted from a Git experiment source code repository and an MLflow AMS instance, we generate provenance graphs according to a graph-based provenance model compliant with W3C PROV and tailored to the semantics of Git and MLflow. The provenance model captures the provenance of ML pipelines, experiments, and their artifacts with respect to the activities in the corresponding Git repository and MLflow instance. Our tool MLflow2PROV¹ performs the provenance extraction and provenance graph generation. Using prepared integrations, it is possible, for example, to import an extracted provenance graph into a graph database and to query for provenance knowledge (see Fig. 1).

This paper is a revised and extended version of [22]. Specifically, we make the following contributions:

- We extend the expressiveness and utility of our provenance model by adding (i) a new submodel that provides an extensive representation of pipeline runs, (ii) additional submodels that cover further activities (e.g., registered model deletion) that were previously practically undetectable, and (iii) entities and attributes to existing submodels (e.g., dataset artifacts).
- We design and implement an approach for capturing end-to-end provenance traces by MLflow2PROV. That now enables the tracking of pipeline artifacts from their creation through all changes to their eventual deletion. By continuously capturing provenance-related events in the MLflow tracking and model registry components at the time of their actual occurrence, continuous and instant updates to the corresponding provenance graph are possible.
- We present an extensive evaluation consisting of two parts: First, the evaluation of the provenance model's expressiveness and utility based on queries representing typical provenance analysis questions and compares the range of supported queries with MLflow and DLProv as a direct competitor. Second, the evaluation of the runtime performance of our tool regarding provenance extraction and graph generation.

The paper is structured as follows: First, we introduce two example ML pipelines, MLflow, and W3C PROV (Section 2). In Section 3, we present our PROV-compliant model capturing the provenance of

collaborative ML model and pipeline development projects including its extensions. Subsequently, we describe the workflow, design, and implementation of our tool MLflow2PROV (Section 4). In Section 5, we present the evaluation and give insights into expressiveness and utility of our provenance model and the runtime performance of MLflow2PROV. Finally, we discuss related work (Section 6) and conclude the paper (Section 7).

2. Preliminaries

This section introduces two ML pipelines, a classification and a regression pipeline, as running examples (Section 2.1), describes preliminaries regarding MLflow (Section 2.2), and gives an overview of the W3C PROV standard and the relevant parts of its data model (Section 2.3).

2.1. ML pipeline examples

We use two pipelines as examples throughout the paper. The first ML pipeline addresses a classification problem using a wine quality dataset [23,24], aiming to create a model that predicts whether a wine is red or white based on its physicochemical properties. The second ML pipeline tackles a linear regression problem with taxi trip data from the Taxi and Limousine Commission (TLC) [25]. Its objective is to predict the fare amount for a taxi ride in New York City, considering factors such as pickup and drop-off locations, trip duration, distance, and more.

Both pipelines follow a common structure (see Fig. 2): (i) The ingestion step loads the specified raw source data. (ii) The split step splits the ingested dataset into a training dataset for model training, a validation dataset for model performance evaluation and tuning, and a test dataset for model performance evaluation. (iii) The transformation step uses the training dataset to fit a transformer that performs the defined transformations. The transformer is then applied to the training dataset and the validation dataset, creating transformed datasets that are used by subsequent steps for estimator training and model performance evaluation. (iv) The training step uses the transformed training dataset to fit an AutoML or user-defined estimator. The estimator is then joined with the fitted transformer to create a model. Finally, this model is evaluated against the transformed training and validation datasets to compute performance metrics. (v) The evaluation step evaluates the

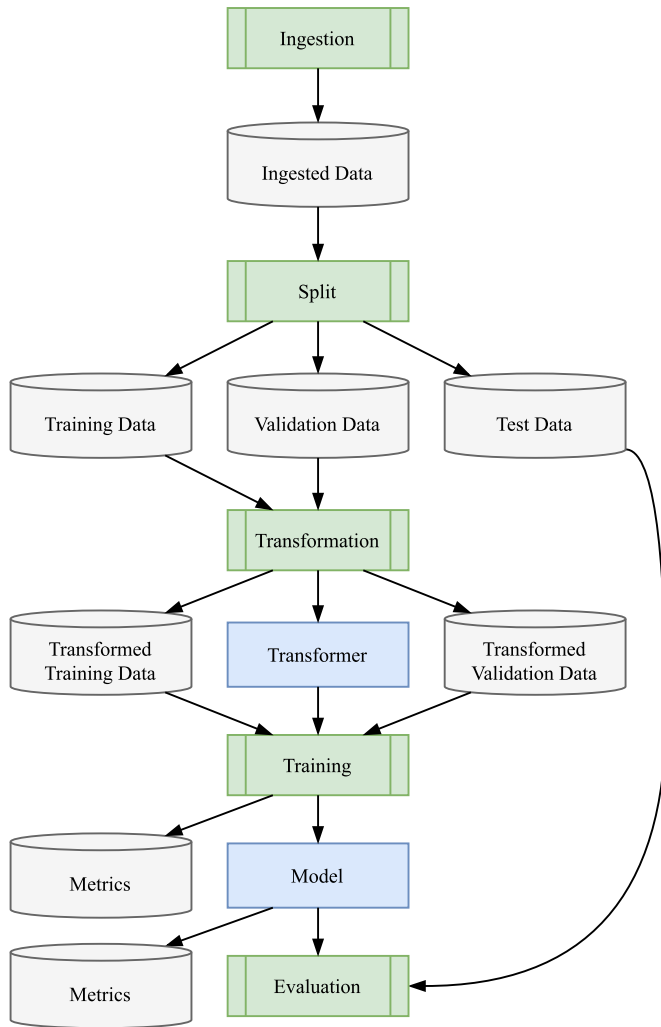


Fig. 2. Pipeline structure of the example classification and regression pipelines.

trained model on the test dataset, computing performance metrics and model explanations. Resulting performance metrics are compared against defined thresholds indicating whether a subsequent iteration through the transformation and training steps is necessary.

2.2. MLflow

MLflow is an open-source platform for ML artifact management focusing on model development and deployment. It provides a suite of tools aiming at simplifying the ML workflow by streamlining cumbersome logging, organization, and lineage concerns, particularly by MLflow Tracking, the MLflow Model Registry, and MLflow Recipes.

MLflow Tracking provides an API and UI for logging data, model, and metadata artifacts (such as parameters, metrics, and code versions) during experiment and pipeline runs, either via automatic logging or manual instrumenting of application code. MLflow Tracking is organized around the concept of runs, which are executions of some piece of code [9]. Each run records metadata, i.e., various information about a run such as metrics, parameters, start and end times, and artifacts, i.e., output files such as model weights, images, etc. Runs can be organized into experiments, that group runs belonging to the same task.

The MLflow Model Registry is a model store for versioning models and managing their full lifecycle. An MLflow Model is created from a run and can then be registered with the Model Registry. A registered model has a unique name, contains versions, aliases, tags, and other

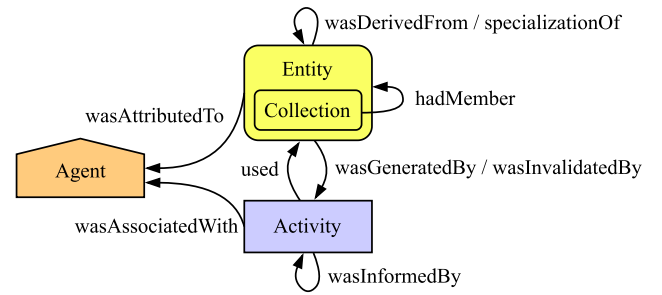


Fig. 3. Overview of relevant W3C PROV concepts.

metadata. Each registered model can have one or many versions, starting with version 1 and incrementing with each new registered model with the same name. Model versions have tags, which is useful for tracking attributes of a model version.

With MLflow 2.0, the MLflow Recipes framework was integrated. This framework simplifies the implementation of ML pipelines through predefined templates for common ML tasks, conventions (e.g., folder and file structure), and tool support (e.g., command line tool, data profiling, and hyperparameter tuning). MLflow Recipes comprises the following relevant concepts: A step represents an individual ML operation, such as ingesting data, fitting an estimator, or evaluating a model against test data. Each step accepts a collection of well-defined inputs and produces well-defined outputs according to user-defined configurations and code. Thus, a recipe is an ordered composition of steps used to solve an ML problem.

2.3. W3C PROV

The W3C PROV standard [11] defines PROV-DM [26] – a generic provenance data model that allows domain- and application-specific representations of provenance to be expressed and interoperably exchanged. PROV-DM defines the following structures relevant to this work (see Fig. 3): (i) types *entity* (Entity), *activity* (Activity), and *agent* (Agent) that are involved in producing a piece of data or artifact, and (ii) relations that relate entities, activities, and agents, such as:

- *wasGeneratedBy*: the production of a new entity by an activity,
- *wasInvalidatedBy*: the destruction or expiry of an existing entity by an activity,
- *used*: the utilization of an entity by an activity,
- *wasInformedBy*: the exchange of some entity by two activities, where one activity uses some entity generated by the other,
- *wasDerivedFrom*: the transformation of an entity into another one, update of an entity resulting in a new one, or creation of a new entity based on an existing entity,
- *specializationOf*: the specialization of a general entity by a more specific one,
- *hadMember*: the belonging of an entity to a collection entity,
- *wasAttributedTo*: the attribution of an entity to an agent, and
- *wasAssociatedWith*: the assignment of responsibility for an activity to an agent.

Capturing the provenance of entities, such as source code files, experiment runs, or any other type of artifact, as well as their interrelations produces a directed acyclic graph (DAG). All nodes and edges of this DAG have defined semantics, thus, resulting in a specific knowledge graph. Consequently, for specific uses of PROV-DM, each type (i.e., entity, activity, and agent) has at least one specialization with application-specific semantics.

3. Provenance model

Developing an ML model typically requires a lot of experimentation, for example with different data preprocessing methods, feature transformations, model architectures and algorithms, and parameter combinations. Thus, the corresponding experiment pipeline is subject to numerous changes and adjustments. The goal of our graph-based provenance model is to represent the changes and executions of an ML pipeline so that instances of the provenance model, i.e., concrete provenance knowledge graphs, are expressive for diverse analysis questions. These can range from parameter- and metadata-related questions, e.g., “Which hyperparameters were used for a given model?”, pipeline-related questions, e.g., “What feature transformations have been applied?”, to composed questions also involving the knowledge of Git-related activities. Some examples are:

- “In which version of a pipeline did metric M improve as a result of changes?” – This type of question helps to identify particularly promising pipeline versions that could be used as potential candidates.
- “Which changes to a pipeline have improved or worsened metric M the most?” – This type of question helps to find important changes and potential bottlenecks.
- “Who created the model with the best performance regarding metric M , at what time, and who contributed the most to that model?” – This type of question helps to identify responsible developers.

This section introduces our provenance model compliant with W3C PROV, which captures the provenance of ML experiments, pipelines, and their artifacts based on Git-related and MLflow-related activities. Inspired by previous work [21,22], our provenance model is composed of several submodels that capture the mentioned effects from Git-related and MLflow-related activities. Types and relations of the submodels include various attributes which are essential for later querying, such as:

- predefined attributes, e.g., `prov:role` and `prov:type`,
- entity attributes, e.g., `name`, `description`, `createdAt`, and `lastUpdatedAt` of the `RegisteredModel` [Entity](#),
- activity attributes, e.g., `prov:startTime` and `prov:endTime` of the `*Creation` [Activity](#), and
- agent attributes, e.g., `name`, `email`, and `username` of the `*Author` [Agent](#).

In most of the following figures, attributes are omitted for clarity.

3.1. Git-related provenance submodels

Three submodels capture the effects that Git commits have on files’ status and contents [27]: the creation of a new file (see Fig. 4), the change of a file (see Fig. 5), and the deletion of a file (see Fig. 6). A commit, that adds a file, results in the following provenance information being captured:

- The file and the file revision at the point of creation (current file version) are represented as `File` [Entity](#) and `FileRevision` [Entity](#). `FileRevision` is a specialization of `File`.
- The commit is represented as `Commit` [Activity](#), which generated both `File` and `FileRevision`. The previous commit (`ParentCommit` [Activity](#)) is also directly linked (`wasInformedBy`).
- The commit author and committer are represented as `CommitAuthor` [Agent](#) and `Committer` [Agent](#). `File` and `FileRevision` are attributed to `CommitAuthor` representing the responsibility for her content. Both agents, `CommitAuthor` and `Committer`, are responsible for that commit and, thus, are associated with `Commit`.

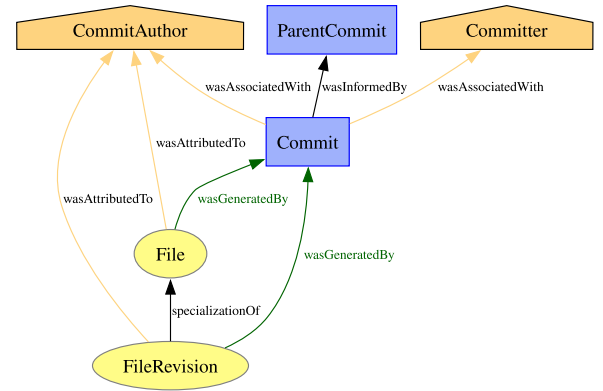


Fig. 4. Submodel representing the creation of a file.

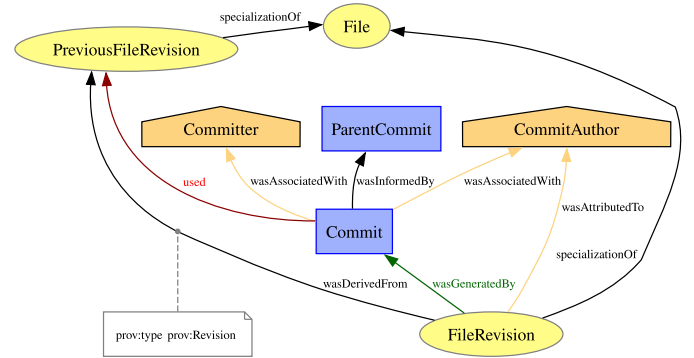


Fig. 5. Submodel representing the modification of a file.

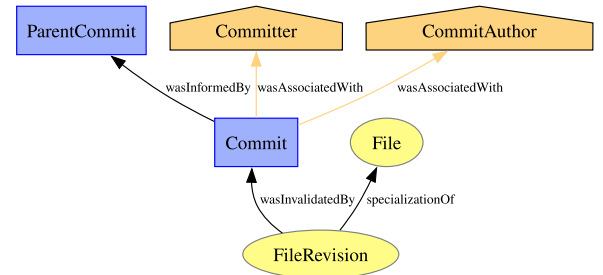


Fig. 6. Submodel representing the deletion of a file.

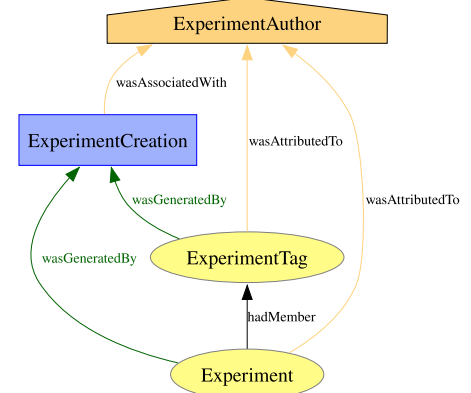


Fig. 7. Submodel representing the creation of an experiment.

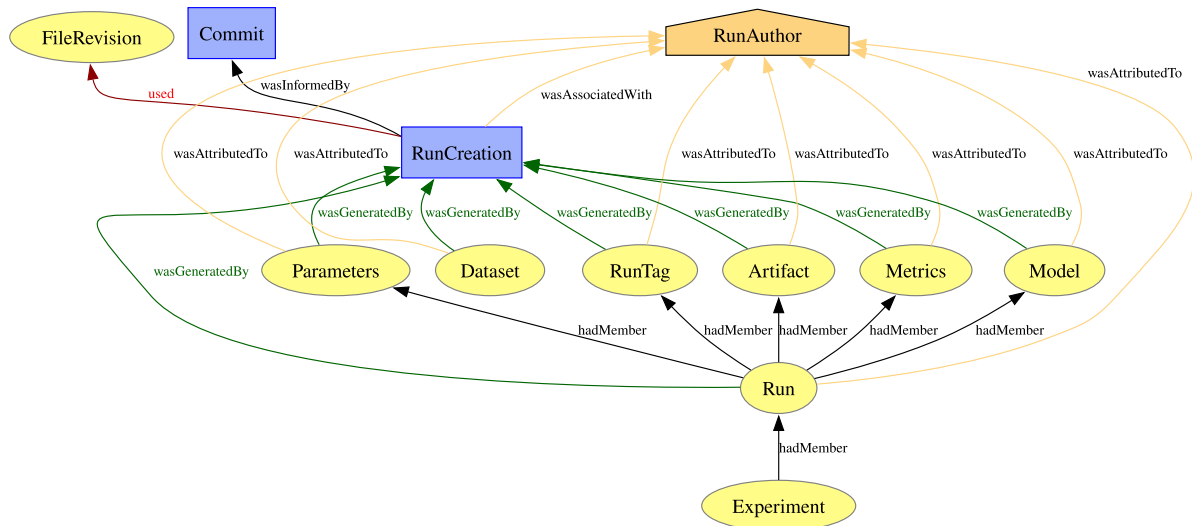


Fig. 8. Submodel representing the creation of a run.

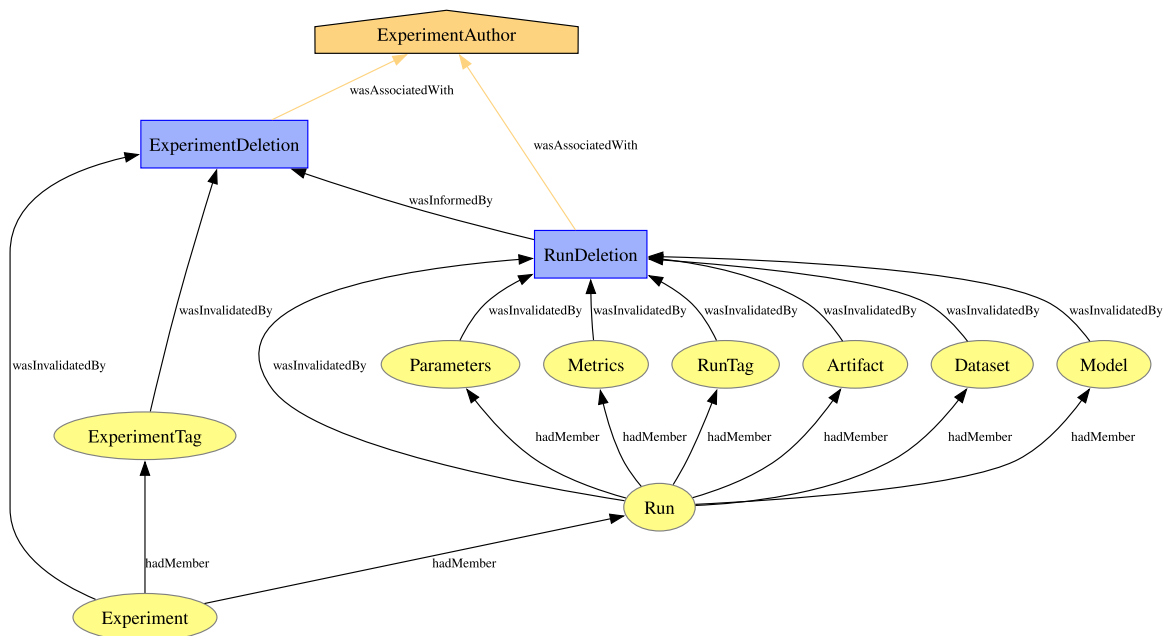


Fig. 9. Submodel representing the deletion of an experiment.

A commit modifying a file has the following differences regarding the captured provenance information:

- Since Commit used PreviousFileRevision (Entity) of File, the newly generated FileRevision (Entity) is derived from PreviousFileRevision. Thus, FileRevision is also a specialization of File.

The deletion of a file by a commit leads to the following provenance information:

- The current FileRevision is invalidated by Commit.

3.2. MLflow-related provenance submodels

Four submodels capture the effects of the creation of a new experiment (see Fig. 7), the creation of a new run (see Fig. 8), the deletion of an experiment (see Fig. 9), and the deletion of a run (see Fig.

10). While the provenance information collected for the creation of an experiment (organized around the ExperimentCreation (Activity)) is represented straightforward and comparable to the file creation model, the run creation model is more comprehensive and has some noticeable characteristics:

- RunCreation (Activity) uses FileRevision (e.g., model training code in train.py) to execute a run and, thus, was informed by the corresponding Commit. RunCreation generates Run (Entity), which is a collection entity, and Metrics (Entity), Parameters (Entity), RunTag (Entity), Dataset (Entity), Model (Entity), and Artifact (Entity), which are members of Run.^{2,3} In addition, the

² The Artifact (Entity) represents any other additionally logged artifacts, such as plot images or log and statistics files.

³ Instead of creating a separate entity for each model parameter and each metric, only a single (Entity) is created for each due to compactness and efficiency.

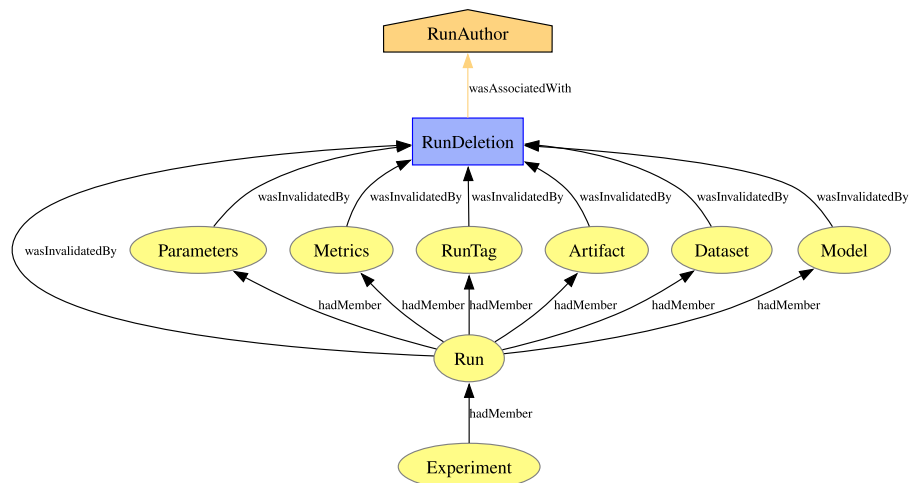


Fig. 10. Submodel representing the deletion of a run.

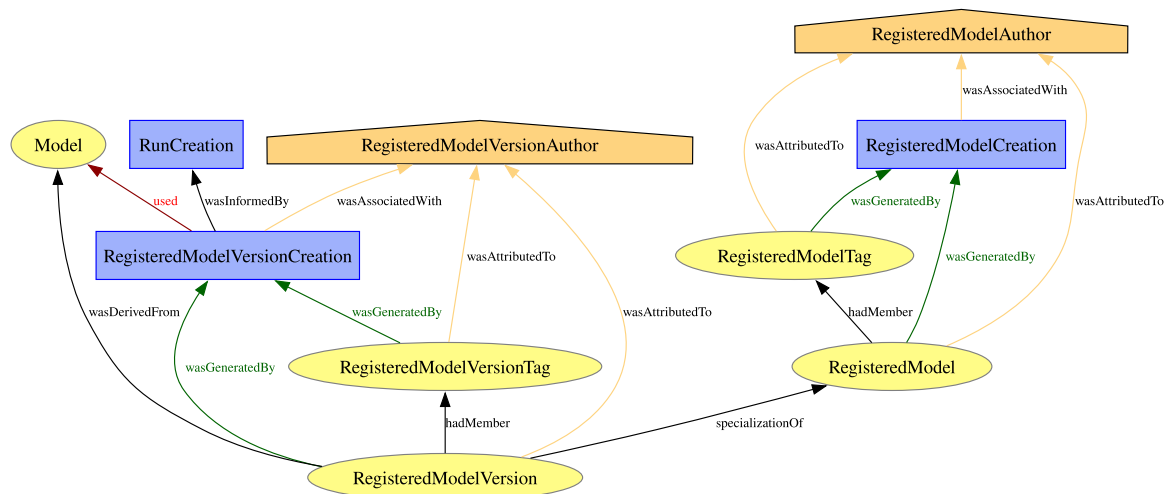


Fig. 11. Submodel representing the creation of a registered model.

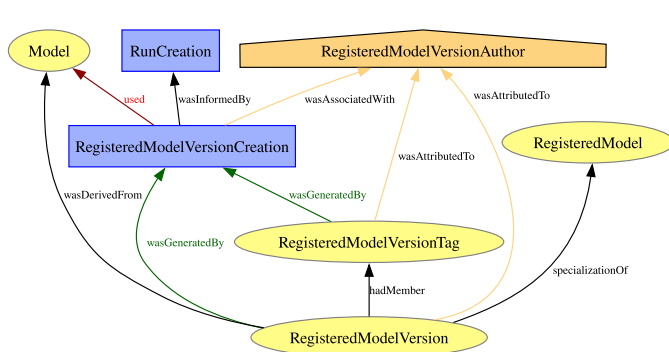


Fig. 12. Submodel representing the creation of a registered model version.

membership relation between `Run` and, if specified, the corresponding `Experiment` is captured. The remaining relations are analogous to the previous submodels.

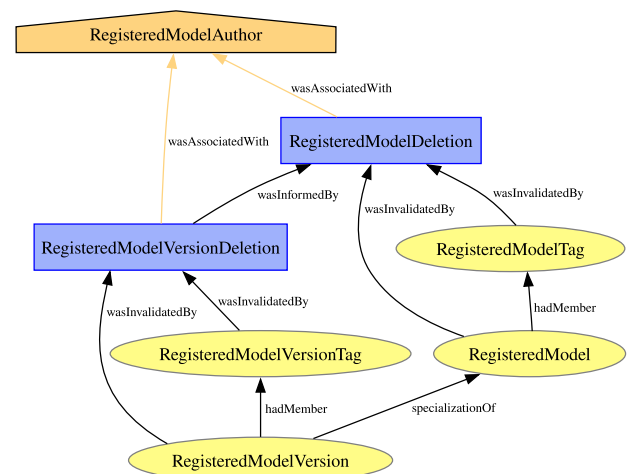


Fig. 13. Submodel representing the deletion of a registered model.

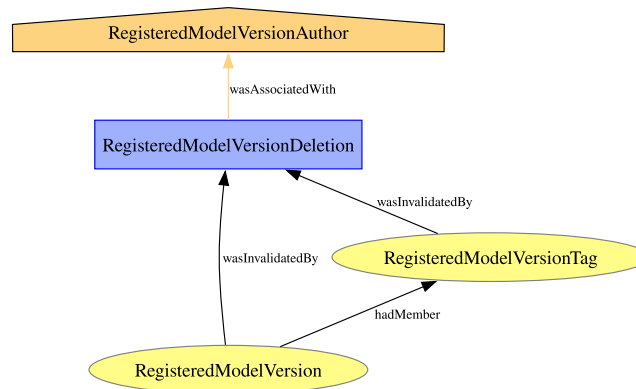


Fig. 14. Submodel representing the deletion of a registered model version.

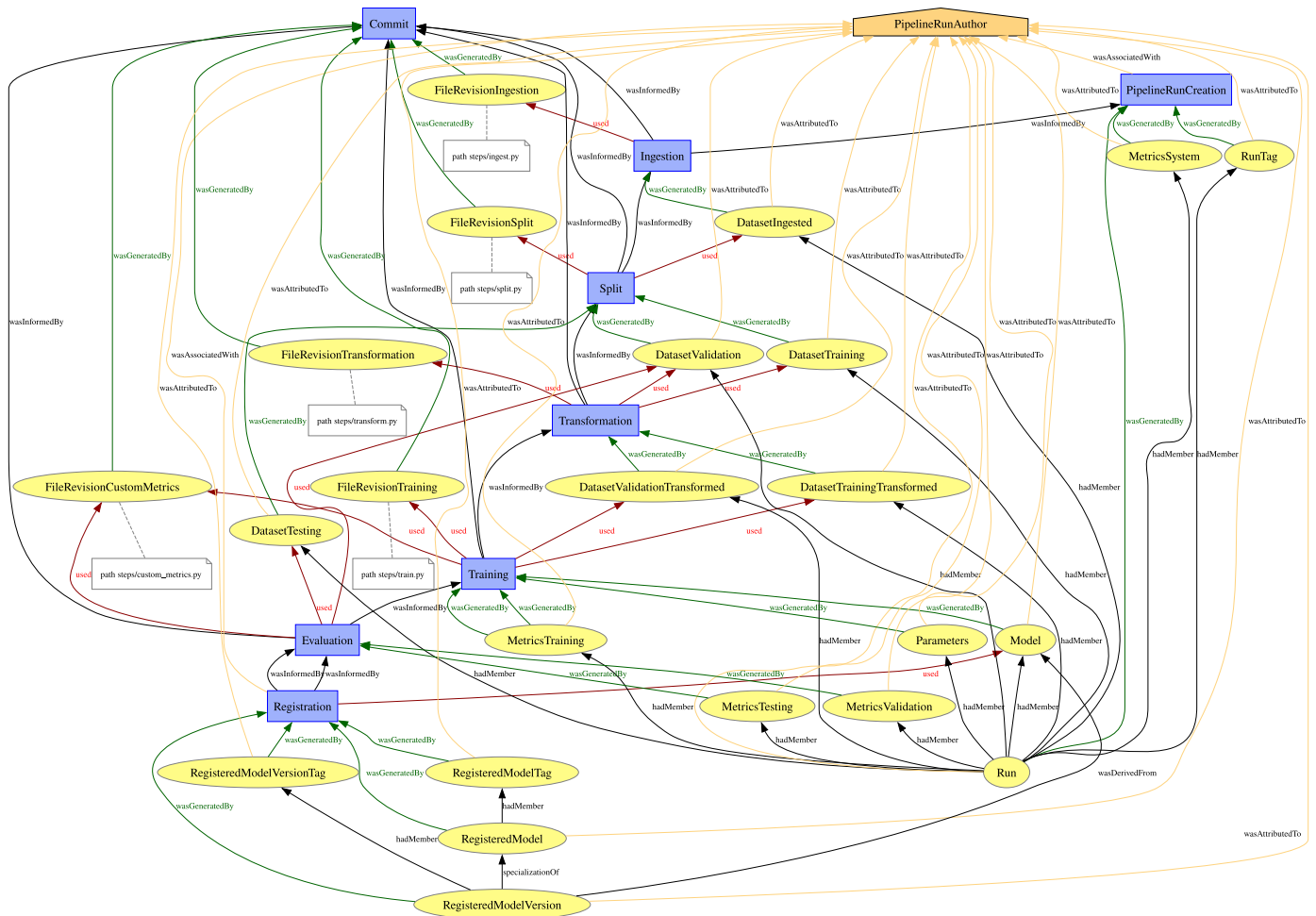


Fig. 15. Submodel representing the creation of a pipeline run.

Moreover, the submodels for deleting experiments (see Fig. 9) and runs (see Fig. 10) are similar to the file deletion model. Some differences are:

- All members of the two collection entities are deleted as well, whereas ExperimentDeletion also includes all associated runs and their members. In the experiment deletion submodel, RunDeletion is triggered by ExperimentDeletion (wasInformedBy relation) for each run assigned to the corresponding experiment.

- Both activities are associated with the author (ExperimentAuthor), who originally created the experiment, as typically only the author has the permissions to delete the experiment.

Besides tracking, MLflow provides a model registry, which is a centralized model store for selected model artifacts (registered models) with support for model lineage, model versioning, stage transitions (e.g., from staging to production), and annotations. The provenance submodels for registered model creation (see Fig. 11), registered model

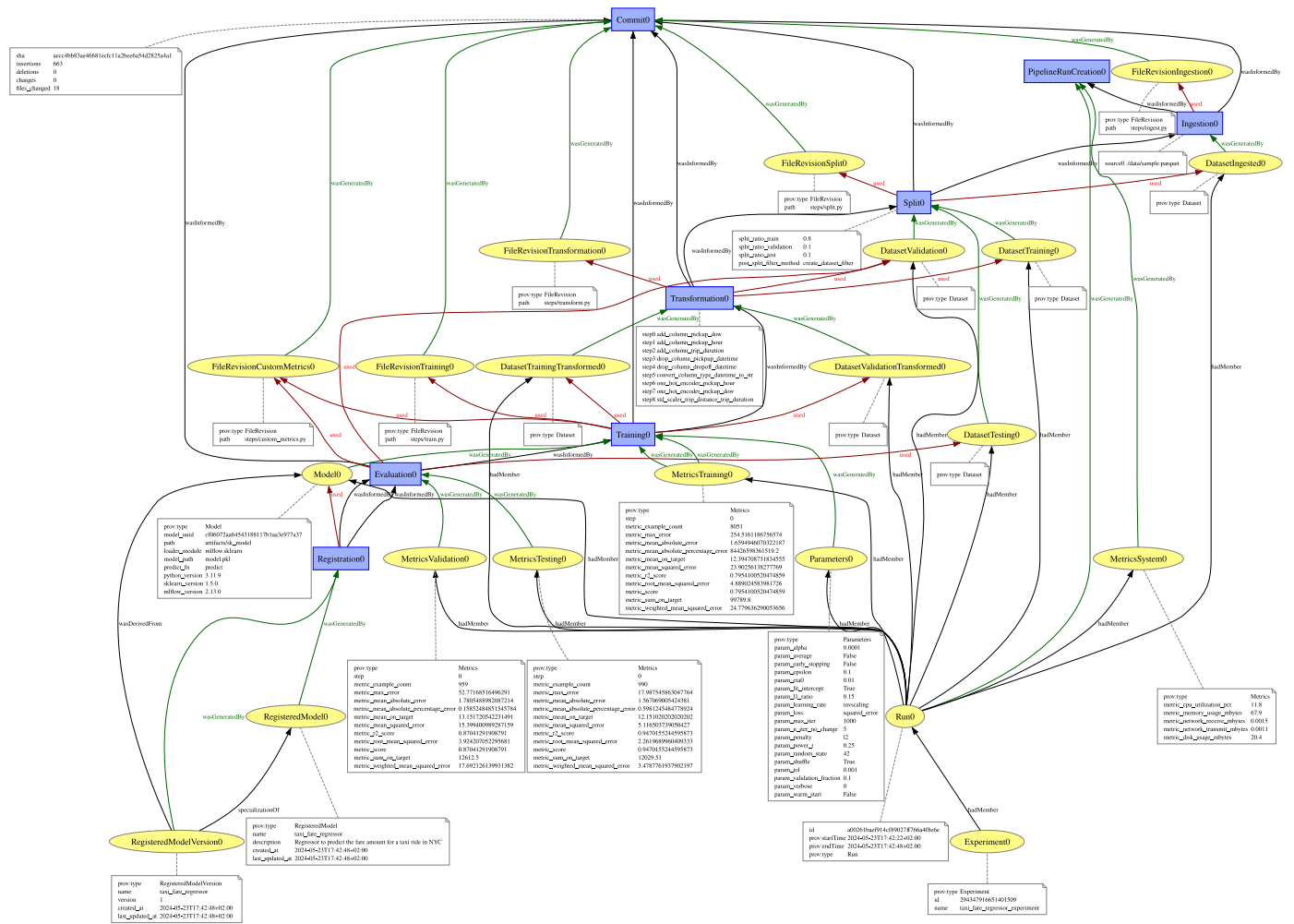


Fig. 16. Instance of the pipeline run submodel representing an execution of the example regression pipeline from Section 2.1 (due to space reasons, only selected attributes are shown, and tag entities and agents are omitted).

version creation (see Fig. 12), registered model deletion⁴ (see Fig. 13) and registered model version deletion (see Fig. 14) are analogous to the models already discussed.

MLflow Recipes supports for the implementation of ML pipelines (see also Section 2.2). It is based on the concepts of steps and recipes: A step represents a pipeline operation, takes a set of inputs, and produces a set of outputs based on user-specified configurations and code. A recipe is an ordered composition of steps to solve a specific ML problem. From the perspective of MLflow's tracking and model registry components, there are no dedicated abstractions for recipes and steps, and no dedicated API for their retrieval. A pipeline run is logged like a "normal" run. However, the artifacts of pipeline runs are stored in a directory structure (using the filesystem as artifact storage backend) so that they can be related to the steps.

The provenance submodel for an MLflow Recipe run, or generally a pipeline run, is built around the PipelineRunCreation Activity and comprises the following provenance information (see Figs. 15 and 16):

- Each pipeline step is represented by an activity: Ingestion Activity, Split Activity, Transformation Activity, Training Activity, Evaluation Activity, and Registration Activity. The steps are linked according to their execution order via wasInformedBy relations, whereby first the Ingestion Activity was informed by the PipelineRunCreation Activity. Except for Registration, each step uses source code that controls the execution of the step (e.g., transform.py represented as the FileRevisionTransformation Entity for the Transformation Activity), which, thus, was informed by the corresponding Commit Activity and working inputs (e.g., DatasetTraining Entity and DatasetValidation Entity for the Transformation Activity), generated by the previous Split Activity and generates outputs (e.g., DatasetTrainingTransformed Entity and DatasetValidationTransformed Entity for the Transformation Activity).
- PipelineRunCreation generates the Run Entity. Analogous to the run creation submodel (see Fig. 8), all entities generated within the ingestion to evaluation steps are members of the Run Entity (hadMember relation).
- After successfully passing the model evaluation step, the corresponding model is registered as a new version of that model (Registration Activity). The captured provenance is analogous to the registered model creation submodel (see Fig. 11).
- Any activities executed in the pipeline and any artifacts generated in the pipeline are related to the executor of the pipeline, the PipelineRunAuthor Agent (wasAttributedTo or wasAssociatedWith).

⁴ This model does not exist in [22] because the deletion of a registered model cannot be easily traced. For example, using the file store backend, the entire model directory including all model versions is irrevocably deleted, making it impossible to identify it retrospectively. We tackle this challenge with the newly introduced command track in combination with an MLflow plugin (see also Section 4.2).

- In addition to the structure itself, i.e., the entities and relationships, the available attributes are crucial to execute specific and meaningful queries. An instance of this model representing an execution of the example regression pipeline from Section 2.1 with a relevant selection of attributes is depicted in Fig. 16. Particularly notable are the numbered step attributes of the Transformation [Activity](#), which allow insights regarding the feature transformations applied to the training dataset and validation datasets.

Recipe executions are not represented internally in MLflow. However, the execution of a recipe typically triggers the creation of a run and its members, a registered model, and a registered model version in the tracking and model registries; and likewise their provenance counterparts. The latter ensures compatibility with the existing sub-models, which already cover the deletion of runs, registered models and registered model versions as explained above (see Figs. 9, 10, and 14).

4. MLflow2PROV

This section introduces our tool MLflow2PROV that extracts provenance information from the VCS and AMS resources of ML model development and experiment projects and generates provenance graphs according to our introduced model. First, Section 4.1 explains the underlying workflow of MLflow2PROV. Subsequently, Section 4.2 describes the tool's design and implementation.

4.1. Workflow

MLflow2PROV provides a command line interface (CLI) that takes a chain of CLI commands and corresponding options (comparable with piped shell commands). Alternatively and functionally equivalent, a .yaml file specifying the commands and their options can be used for configuration. The following commands are available:

- **extract:** This command extracts provenance information from the corresponding resources, using paths and URLs of the associated Git repositories and MLflow instances, and produces a corresponding provenance graph for each.
- **load:** This command loads PROV files (e.g., .rdf, .json, or .xml) containing provenance graphs from previous extractions (e.g., for updating them).
- **save:** This command saves each extracted provenance graph in a separate file according to the specified format (e.g., .rdf, .json, .xml, .provn, or .dot).
- **merge:** This command merges multiple provenance graphs, which originate from the extraction from multiple sources, into a common one.
- **transform:** This command applies various transformations, such as the merging of multiple agents belonging to the same person based on user-defined mappings, the elimination of duplicates, and the replacement of user-related information (e.g., usernames and email addresses) with pseudonyms in the given provenance graphs.
- **statistics:** This command prints statistics related to the given provenance graphs (e.g., number of PROV elements and PROV relations for each type).
- **track:** This command switches on continuous provenance tracking, which collects provenance information continuously after the initial extraction (extract) and/or load (load) and updates the corresponding provenance graph.

By calling MLflow2PROV with the **extract** command, a provenance graph is generated for each project based on the given Git repositories and MLflow instances. This is accomplished in three major steps (see Fig. 17):

1. First, each given Git repository is cloned if the user specified a URL instead of a local path. Following this, a `git log` command is invoked per repository, the output is parsed, and converted to intermediate data structures that represent Git-related entities such as commits, files, users, etc.
2. Next, the given MLflow servers are queried via the MLflow APIs retrieving stored metadata of pipelines, experiments, runs, and artifacts (models, parameters, metrics, etc.) and translated into intermediate data structures.⁵
3. Finally, the intermediate data structures are converted into PROV-compliant data structures, which are used to generate the provenance graph associated with each project. In addition, post-processing steps are performed according to further commands (e.g., **transform** and **merge**) as well as the output of the PROV document in the selected formats (**save**).

Typically, this initial extraction is supplemented by continuous extraction using the **track** command: Provenance-relevant events within the MLflow tracking store and the MLflow model registry are detected using our own MLflow plugin. The necessary information are transmitted to MLflow2PROV via its REST API for continuous tracking. That immediately triggers the execution of the above steps for provenance information extraction and provenance submodel graph generation. If a previously unknown commit ID is found (e.g., in a run's metadata), the execution of `git fetch` on the repository is triggered, as well as the provenance information extraction and generation of the corresponding provenance submodel graphs.

The exported provenance graph in the output PROV document can be imported for processing and querying into specific provenance storage systems (e.g., ProvStore [13]) or graph database systems (e.g., Neo4j, queryable with Cypher), or visualized (e.g., with Plotly). For example, using the Neo4j integration (see also our GitHub repository⁶), the question of “*Who created the model with the best performance regarding metric accuracy, at what time, and who contributed most to that model?*” can be answered with the result of the Cypher query from Fig. 18.

4.2. Design and implementation

MLflow2PROV is implemented in the Python programming language using libraries for MLflow, Git, and W3C PROV. In the following, we focus on the software architecture and the implementation approach for the provenance extraction.

Software architecture. We adapt architecture design patterns that follow domain-driven design principles, allow managing application complexity, and have been established for Python software [28]. Fig. 19 provides an overview of MLflow2PROV's general software architecture.

According to the *Domain Model Pattern*, our provenance model (see Section 3) forms our domain model. It is implemented in a separate subpackage and independent of the other layers of MLflow2PROV, which eases testing and refactoring, since our domain model is not tangled up with the other layers. By applying the dependency inversion principle [29], everything else is built around the domain model.

The *Repository Pattern* provides an abstraction over data storage decoupling the Domain Model Layer from the Database Layer, which stores the extracted provenance information. Moreover, we apply the *Service Layer Pattern* to take care of orchestrating our workflows and defining MLflow2PROV services for fetching and extraction, processing, as well as provenance graph creation and serialization. This way, the Service Layer is the entry point to our domain model and the CLI as the Presentation Layer exclusively talks to the Service Layer.

⁵ Further EMSs can be supported by adapting or extending intermediate data structures and implementing additional procedures for extracting from EMS APIs.

⁶ <https://github.com/mariusschlegel/mlflow2prov/>

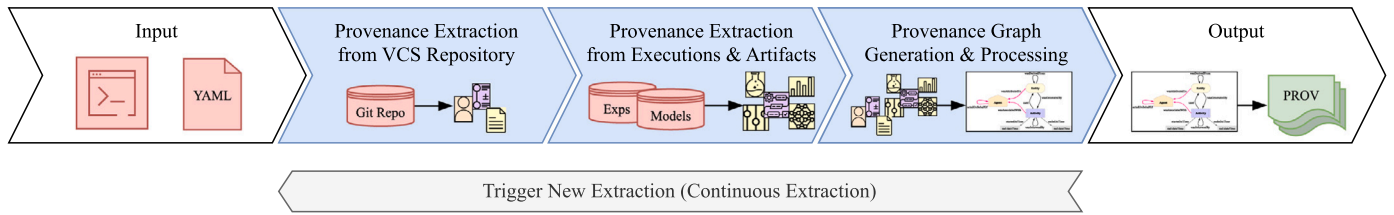


Fig. 17. The workflow underlying MLflow2PROV. The major steps for provenance extraction and graph creation are highlighted.

```
CALL {
  MATCH
    (run:Entity)-[:hadMember]-(metrics:Entity),
    (run:Entity)-[:hadMember]-
      → (runTagUser:Entity),
    (run:Entity)-[:hadMember]-
      → (runTagSourceName:Entity),
    (run:Entity)-[:wasGeneratedBy]-
      → (runCreation:Activity)

  WHERE
    metrics.`prov:type`="Metrics" AND
    runTagUser.`prov:type`="RunTag" AND
    runTagUser.name="mlflow.user" AND
    runTagSourceName.`prov:type`="RunTag" AND
    runTagSourceName.name="mlflow.source.name"

  RETURN
    run.run_id AS model_run_id,
    metrics.metric_accuracy AS accuracy,
    runTagUser.value AS created_by,
    runCreation.`prov:startTime` AS created_at,
    runTagSourceName.value AS run_source

  ORDER BY
    accuracy DESC LIMIT 1
}

MATCH
  (commit:Activity)-[:wasGeneratedBy]-
    → (fileRevision:Entity),
  (user:Agent)-[:wasAssociatedWith]-
    → (commit:Activity)

WHERE
  commit.`prov:type`="Commit" AND
  fileRevision.name=run_source AND
  fileRevision.`prov:type`="FileRevision"

WITH
  model_run_id,
  accuracy,
  created_by,
  created_at,
  run_source,
  user.name AS contributed_most,
  COUNT(DISTINCT commit) AS commit_count

RETURN
  model_run_id,
  accuracy,
  created_by,
  created_at,
  contributed_most

ORDER BY
  commit_count DESC
```

Fig. 18. Cypher query that determines the model that maximizes the metric *accuracy*, the creator and the creation date (left column, subquery), and the user who contributed most to the pipeline's training step (right column).

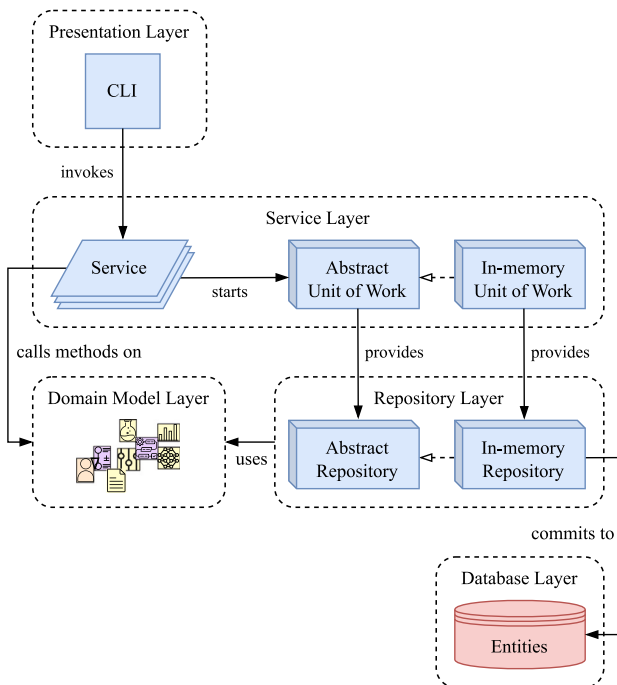


Fig. 19. The software architecture of MLflow2PROV.

For tying together the Repository and Service Layers, we apply the *Unit of Work (UoW) Pattern*. This pattern provides an abstraction over data access by representing atomic operations. It enables decoupling the services within the Service Layer from the Repository and Database Layers. Each service runs in a single UoW (as part of the Service Layer) that is responsible for accessing the Repository and Database Layers and succeeds or fails as a block.

Provenance extraction. MLflow enables several setups by allowing the choice of backends for tracking metadata and artifact storage: In team development scenarios, a remote tracking server together with a separate relational database system server for MLflow metadata and an object storage service for artifacts can be used. Solo developer scenarios use a local, SQLAlchemy-compatible database (e.g., SQLite, PostgreSQL, or MySQL) for storing tracking metadata and the local filesystem as artifact storage or, even simpler, the filesystem for both without setting a dedicated database. Independently of the used backend, the metadata required for creating provenance submodel instances can be retrieved via the MLflow Python API. However, for our context, this has the limitation that only the latest state is accessible; the complete history cannot be retrieved retrospectively. To continuously capture all provenance-relevant events, the command `track` keeps MLflow2PROV running after the initial provenance extraction and enables continuous extraction and graph updates.

There are two implementation approaches for this: (i) The commercial version of MLflow hosted by Databricks provides predefined webhooks. These allow listening to events and triggering actions. While

this approach is non-intrusive and does not require to MLflow, the webhooks cover only some required information and events in the model registry, but not those in the tracking registry. In addition, the webhooks are not available in the open source version of MLflow. (ii) The integration of hooks into the MLflow backend implementations (file store, SQLAlchemy store, REST store, etc.) makes it possible to effectively and efficiently detect relevant events and forward relevant information to MLflow2PROV. Although this approach has a high degree of flexibility, it requires the modification of the existing backend implementations. A less intrusive way is provided by MLflow itself through its plugin mechanism, which we have realized for the file store and the SQLAlchemy store.

5. Evaluation

In this section, we present the evaluation consisting of two parts: (i) the evaluation of our provenance model's utility based on queries representing typical provenance analysis questions and compares the range of supported queries with MLflow and DLProv [19] as a direct competitor (Section 5.1), and (ii) the evaluation of MLflow2PROV's runtime performance that considers the provenance extraction and the graph generation (Section 5.2).

5.1. Model utility and query support

Model utility and query support is evaluated in comparison to pure MLflow and DLProv. The MLflow UI makes it possible to display logged artifacts from experiments and runs as well as registered models and model versions. Searching and filtering is possible using a very basic, SQL-inspired domain-specific language (DSL), which allows using comparison operators or membership checks, for example. In addition, MLflow offers a command line interface that can be used to obtain the information about experiments and runs, such as via `mlflow runs describe --run-id <run_id>` which basically prints JSON-formatted text to stdout.

DLProv [19] is a service to enable DL pipeline analyses over a PROV-compliant provenance data representation. DLProv is built on top of the dataflow web tool DfAnalyzer used for recording provenance information via a REST API. While DLProv is independent of ML frameworks, it requires manual instrumentation of the target code. Moreover, DLProv uses the DfAnalyzer's integration for MonetDB which enables querying via SQL.

Table 1 lists queries for provenance questions of interest in natural language obtained from the literature. Queries Q1–Q9 aim at tracing the provenance and metadata of individual ML artifacts, the relations between artifacts, and the execution and steps of pipelines and model experiments. Moreover, queries Q10–Q13 target the development history (evolution of pipeline and experiments). We compared the support of the queries with pure MLflow and the direct competitor DLProv. The results can be found in Table 2.

Q1 and Q2 require tracing the dataset used for model training. MLflow captures this information automatically when using MLflow recipes, or manually by using the `log_input` operation. DLProv follows the data derivation path from the trained model (`iTesting` Entity) that was generated by a `Training` Activity that used a set of data identified by an `oTrainingTuples` Entity. Since MLflow2PROV uses provenance traces and artifacts logged by MLflow, it also captures any datasets, such as raw ingested datasets or training, validation, and test datasets (`Dataset` Entity) of the corresponding `prov:type`, which can be followed through the `wasGeneratedBy` relations to the corresponding pipeline step (e.g., `Ingestion` Activity, `Split` Activity, or `Transformation` Activity) or the membership relation to the corresponding `Run` Entity.

Q3 can be answered through the MLflow UI by checking the hyperparameter in the “Overview” tab of a run. To answer Q3, DLProv considers the entity related to the hyperparameters, that were used in

Table 1

Typical ML provenance queries in natural language.

ID	Query Description
Q1	Which data(set) was used to train a given model, and what is its origin?
Q2	Which dataset was used for computing the evaluation data?
Q3	Which hyperparameters were used in this model?
Q4	Who was responsible for training a given model?
Q5	What is the hyperparameter configuration used to train the model with the highest average training accuracy?
Q6	What feature transformations have been applied to the data?
Q7	Which models were trained with hyperparameter <i>H</i> and value <i>V</i> ?
Q8	What was the computational environment used to train a given model?
Q9	Which process led to a given model (e.g., the best-performing model)?
Q10	From which revision was given model built?
Q11	Which change set resulted in model degradation?
Q12	How many developers contributed to the pipeline producing a given model?
Q13	Who contributed most to the pipeline producing the best performing model?

Table 2

Query support of DLProv, MLflow, and MLflow2PROV (“●” – supported, “◐” – partially supported, “○” – not supported).

ID	MLflow	DLProv	MLflow2PROV
Q1	●	●	●
Q2	●	●	●
Q3	●	●	●
Q4	◐	●	●
Q5	◐	●	●
Q6	◐	●	●
Q7	●	●	●
Q8	●	●	●
Q9	◐	◐	●
Q10	●	○	●
Q11	○	○	●
Q12	○	○	●
Q13	○	○	●

the `Training` Activity. The graph that represents the outcome of this query follows an `iTesting` Entity that was generated by a `Training` Activity that used an `iTraining` Entity containing hyperparameter attributes. MLflow2PROV can answer Q3 by following the derivation path of the `Parameters` Entity.

Q4 is related to the users (i.e., agents) involved in model training. Although MLflow does not directly capture a user's metadata, it does contain the user's login name of the machine where the training was performed. Both DLProv and MLflow2PROV associate the responsible agent with captured pipeline steps, such as `Training` Activity, enabling answering this question.

For answering Q5, directly averaging a particular metric in the MLflow UI is not easily possible, unless the average value itself is logged as a metric. DLProv or MLflow2PROV can answer Q5 by matching the `Training` Activity, which generated the `oTraining` Entity or `MetricsTraining` Entity with the highest average accuracy, then following the `wasGeneratedBy` relation to the `iTraining` Entity or the `Parameters` Entity containing the target hyperparameter configuration.

Q6 asks regarding applied feature transformations. MLflow logs transformer pipelines as serialized object artifacts (e.g., for scikit-learn in pickle format). Beside displaying the HTML step card of a pipeline's transform step, there is no built-in procedure to query the applied feature transformations (unless they were logged explicitly, e.g., as tags). In DLProv, answering Q6 involves path traversal from the trained model to the activities that an input dataset (e.g., `iInputDataset_101` Entity) used in data preprocessing (e.g., `Operation1_101` Activity). MLflow2PROV is also capable of extracting data preprocessing and feature transformation flows from pipeline runs (e.g., see the attributes of the `Transform` Activity in Fig. 16).

Q7 can be answered with the MLflow UI by filtering the runs in an experiment (e.g., by setting `params.opt_learning_rate = 0.002`). DLProv allows filtering trained models, `iTesting` Entity, that was generated by a `Training` Activity, that used `learning_rate =`

0.002 as training input (iTraining [Entity](#)). In MLflow2PROV, answering Q7 requires following the membership relations between Metrics [Entity](#) and Training [Activity](#), and Training [Activity](#) and Model [Entity](#).

MLflow captures information about the execution (training and inference) environment, such as versions of Python, MLflow, and the ML library used (e.g., scikit-learn) and system metrics (enabled by setting the environment variable MLFLOW_ENABLE_SYSTEM_METRICS_LOGGING). Thus, Q8 can be answered using the MLflow UI's artifacts section and a run's system metrics. DLProv provides a dedicated entity of type computational_environment, where this information can be found. In MLflow2PROV, this information is found in the attributes of a Model [Entity](#) and a MetricsSystem [Entity](#), which are both in a membership relation to the corresponding Run [Entity](#).

Q9 can only be answered very rudimentary in MLflow, i.e., by manually traversing the filesystem directories that contain logged metadata and artifacts; declarative queries, e.g., regarding the relations within a pipeline run, are not possible. Both DLProv and MLflow2PROV can answer this question by traversing their provenance graph: In DLProv, starting with an iInputDataset [Entity](#) following preprocessing operation activities (Operation [Activity](#)) over the subsequent activities to the Testing [Activity](#); in MLflow2PROV, by following the path from the PipelineRunCreation [Activity](#) over the following activities to the Registration [Activity](#).

MLflow records the Git repository URL, branch name, and the commit ID of the source code of the experiment or pipeline run, which can be used to answer Q10. This is not possible in DLProv, as no information on the source of runs is captured. MLflow2PROV extracts the above information from MLflow and links it to the complete history of the associated Git repository.

Q11–Q13 can only be answered with MLflow2PROV, as this is the only one of the systems that also includes VCS repository activities: By considering one or more metrics and their values, the training source code (Training [Activity](#)) used in activities (e.g., FileRevision [Entity](#)) can be used to draw conclusions about the changes made to this file or also about the any changes in the associated commit, and consequently, to determine, which change set resulted in model degradation (Q11). Moreover, the number of developers involved in developing a pipeline producing a particular model can be determined by following the model's derivation path and considering the associations with and attributions to authors (i.e., PROV agents). Q13 can be determined by utilizing known derivation paths. An example of the corresponding Cypher query can be found in Fig. 18.

5.2. Performance evaluation

The evaluation addresses the practical feasibility of our approach for comprehensive provenance extraction and provenance graph generation. More specifically, we aim at the runtime performance of MLflow2PROV. Subsequently, we first describe the evaluation methodology and then discuss the evaluation results.

Methodology. As a practical use cases, we use the two ML pipelines introduced in Section 2.1: First, an ML classification pipeline using a wine quality dataset that creates a model to predict whether a wine is red or white based on its physicochemical properties. Second, an ML regression pipeline that creates a model to predict the fare amount for a taxi ride in New York City. The Git repository and the MLflow server are prepared as inputs in different configurations, ranging from 25 to 200 elements, i.e., commits and runs each, increasing stepwise by 25. The MLflow server uses the file store backend (filesystem) to store experiment and run metadata as well as artifacts. We run the following measurements: (i) We measure the time required for the initial provenance extraction and initial graph generation. (ii) We measure the runtime required to update the provenance graph, which essentially consists of creating a provenance submodel instance (submodel graph) and merging it with the overall provenance graph.

All measurements were performed on notebook hardware with an AMD Ryzen 7 PRO 6850U CPU at 2.7 GHz and 32 GiB LPDDR5 RAM at 6400 MHz. The machine runs Fedora Linux 40 with Linux kernel version 6.9.10. We use Python version 3.12.4 built from the official source release and MLflow version 2.14.3. The runtimes are measured using the high-precision clock from time with nanosecond resolution, which for Linux uses CPU Time Stamp Counter used as clock source. For making the independent execution time measurements more comparable, we temporarily turn off the Python's garbage collector. Moreover, to avoid erratic runtime behavior, we disable dynamic frequency scaling (AMD Turbo Core) and simultaneous multithreading (SMT). To reliably filter outliers, we perform 1000 iterations for each measurement and calculate medians.

Results. First, Fig. 20 shows the results for the measurement of initial extraction and initial graph generation (or one-shot provenance graph extraction, depending on MLflow2PROV's mode of operation). Overall, the results show a linear correlation for both pipelines: While the first measurement configuration with 25 elements (i.e., 25 commits in the associated Git repository and 25 runs and models in the MLflow tracking server and model registry) requires almost 14.5 s, the runtime increases linearly over the measurement with almost 60 s for 125 elements to almost 95 s for 200 elements. At first glance, the runtimes may seem quite long, but this is put into perspective when the use case is examined more closely: Each pipeline run comprises a new registered model version, 21 resp. 19 logged model parameters, 38 resp.

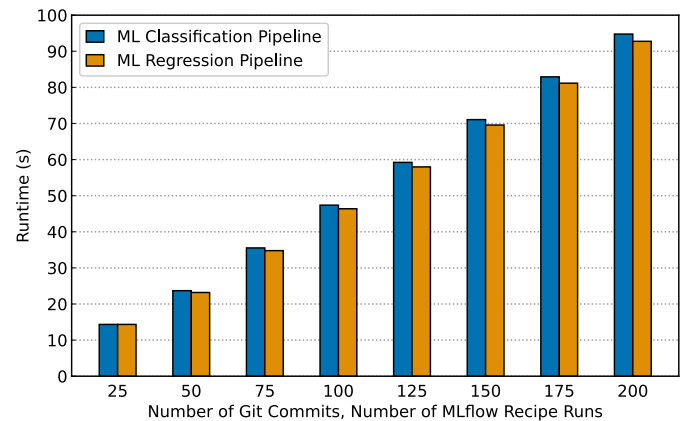


Fig. 20. Measured runtimes for initial provenance extraction and initial graph generation.

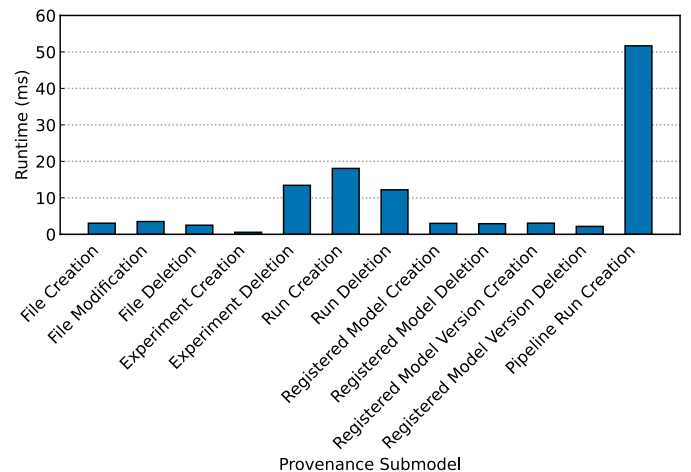


Fig. 21. Measured runtimes for creating provenance submodels with MLflow2PROV.

41 logged training, validation, test and system metrics, 15 tags, and various artifacts for each pipeline step including the training, test and validation dataset metadata, a complete snapshot of the pipeline itself, model step cards, and more.

Second, Fig. 21 shows the results for the measurements of provenance graph updates, which essentially consists of creating a provenance submodel instance (submodel graph) and merging it with the overall provenance graph. The runtimes roughly correlate with scale and complexity of respective provenance submodel and, therefore, can be divided into two classes: While the creation of the submodels for file addition, file modification, file deletion, experiment creation, registered model creation, registered model deletion, registered model version deletion, and registered model version deletion is in the range of up to 3.5 ms, the creation of the submodels for run deletion, experiment deletion, and run creation requires ca. 12.2 ms, ca. 13.4 ms, and ca. 18 ms, respectively. In addition, the multiple of the runtime for the run addition submodel also has an implementation-related reason, as the separate in-memory repository for Git-related resources must also be queried, in addition to the one for MLflow-related resources (see also Section 4.2). Finally, the pipeline run creation submodel is much more extensive than the models considered so far, in terms of the number of entities and activities, as well as the relationships. For this reason, the runtime is also significantly higher, at over 2.5 times the runtime required to generate the run creation submodel.

6. Related work

The intersection between provenance and ML is an increasingly studied area. Our work relates to two subareas: (i) the modeling of provenance of ML pipelines and experiments and (ii) the extraction of provenance from ML pipelines and experiments.

Provenance models for ML experiments and workflows. A recent line of work proposes standardized provenance models for ML workflows and experiments. MEX [16], ML Schema [17], PROV-ML [18], REPRODUCE-ME [30], and RO-Crate [31] are most closely related to our work.

MEX [16] represents basic information of ML experiments by reusing existing ontologies such as PROV-O. Its aim is not to describe a complete ML workflow but to provide a simple and lightweight vocabulary for exchanging ML experiment metadata to achieve a high level of interoperability and supporting data management for ML outcomes. ML-Schema (MLS) [17] is an ontology that represents interchangeable provenance information on ML experiments to foster interoperability and reproducible research. In the OpenML platform [32], MLS is used to export ML datasets, tasks, workflows, and runs as linked open data. PROV-ML [18] is a provenance data representation for the workflow of scientific ML. PROV-ML extends PROV-Lake employs elements of MLS. PROV-Lake extends the W3C PROV data representation and is specialized for workflows that process data in data lakes.

As with other workflow provenance data representations, PROV-Lake alone lacks ML-specific semantics. PROV-ML bridges this gap between workflows and ML concepts by building on PROV-Lake and MLS representations. REPRODUCE-ME [30] is a data model and ontology to describe the end-to-end provenance of scientific experiments. It extends existing standards such as PROV-O and P-Plan. The ontology brings together different aspects of the provenance of scientific studies by interlinking on-computational data and steps with computational data and steps to achieve understandability and reproducibility.

The development of these provenance data representations and ontologies is a significant step towards ensuring the unambiguous interpretability and reproducibility of ML experiments. However, these provenance representations and models have one or more of the following drawbacks: (i) the representation of ML pipelines and ML model development workflows is not considered, as well as the attribution of used and produced artifacts to different pipeline and experiment steps; (ii) the representation of AMS- and MLflow-specific semantics of entities, activities, and their relations is not easily and intuitively possible; (iii) the representation of VCS- and Git-related activities is not considered and, thus, the linkage between pipeline and experiment artifacts and VCS repository artifacts and activities, such as source code and their changes, is not possible; and (iv) the attribution to users or authors of produced and generated artifacts and executed activities is not considered.

RO-Crate [31] is an approach for packaging research data together with their metadata. For a particular domain, these general-purpose containers are constrained to a specific profile – a set of conventions, types, and properties expected to be present [33]. The Workflow Run Crate Working Group has developed three profiles of levels of detail to capture the provenance of execution provenance of computational workflows and scripts. However, these profiles are not tailored to the context and domain of ML pipelines and experimental model development, nor the abstractions and entities of AMSs like MLflow. Our provenance model is based on and derived from W3C PROV which has been an established standard in the provenance community for more than a decade now. A mapping of our PROV-compliant model to RO-Crate is possible through a tool-supported approach using the Simple Knowledge Organization System (SKOS) [34] (see also [35,36]).

Systems and tools for provenance capturing from ML pipelines and experiments. In recent years, a number of systems and tools have been introduced for capturing and extracting provenance information from ML pipelines and model development experiments, such as DLProv [19], DPDS [37], ExperimentTracker [38], mlinspect [39], MLProvLab [40], Vamsa [41], and Geyser [20]. We differentiate these systems below on the basis of the following criteria (see Table 3): (i) the coverage of the typical stages of ML model development workflows (data preparation and preprocessing and model engineering); (ii) the provenance granularity, distinguishing coarse-grained (workflow- and entity-level) and fine-grained (record-level) provenance; and (iii) the provenance

Table 3

Comparison of provenance systems and tools (“●” – property provided/present, “◐” – property partially provided/present, “○” – property not provided/present).

	DLProv	DPDS	ExperimentTracker	mlinspect	MLProvLab	Vamsa	Geyser	MLflow2PROV
Workflow Coverage								
Data Preparation and Preprocessing	●	●	●	●	●	●	●	●
Model Engineering	●	○	●	○	●	●	●	●
Granularity								
Coarse	●	○	●	○	●	●	●	●
Fine	○	●	○	●	○	○	●	○
Capturing								
Framework-agnostic	●	○	○	○	●	◐	◐	●
Non-intrusive	○	●	○	●	●	●	●	◐

capturing with regard to the independence and support of specific data processing and ML frameworks as well as provenance capturing intrusiveness (i.e., whether explicit log statements are required, or not).

DLProv [19] is a provenance data-centric service to support DL workflow analyses over a W3C PROV-compliant provenance data representation. DLProv captures provenance metadata and traces independently of the used framework, but requires explicit log statements. DPDS (Data Provenance for Data Science) [37] allows investigating how operators in data preparation pipelines change the data profile of a dataset through fine-grained explanations. DPDS captures the provenance of individual elements in a dataframe across a series of transformations based on W3C PROV. Focusing on tabular data, it is implemented as a toolkit for Pandas. ExperimentTracker [38] is a lightweight system to extract, store, and manage metadata and provenance information of common artifacts in ML experiments: datasets, models, predictions, evaluations, and training runs. It is integrated with popular frameworks such as MXNet, SparkML, and scikit-learn. Mlin-spect [39] is a Python library that enables provenance-based inspection of ML preprocessing pipelines to find data distribution bugs. Its idea is to extract a directed acyclic graph representation of a pipeline's dataflow and to automatically instrument the code with predefined inspections based on an annotation propagation approach. Mlin-spect operates on declarative abstractions of popular data science libraries like estimator/transformer pipelines and does not require manual code instrumentation. MLProvLab [40] is a JupyterLab extension, to track, capture, compare, and visualize the end-to-end provenance of ML pipelines in notebook code environments. This tool is designed so that users do not need to change scripts or configure with additional annotations. Vamsa [41] focuses on automatically tracking the relationship between data and ML models to aid in model debugging, sharing, compliance, fairness, and explainability. Vamsa implements code-agnostic provenance tracking for data science workflows in Python leveraging static analysis techniques that do not require user modifications or input and are adaptable to new ML frameworks and libraries. Geyser [20] builds upon Vamsa to address the limitations of static provenance extraction. By incorporating runtime information, Geyser enables dynamic, knowledge-base-driven provenance extraction, capturing both coarse-grained and fine-grained provenance. Geyser supports a broad range of data science pipelines and applications, with capabilities for provenance storage and querying.

Each of the systems considered and described has its significance. However, none considers activities within VCS repositories related to the evolutions of ML pipelines and experiments. In contrast, MLflow2-PROV and its underlying provenance model combine high-level provenance (also referred to as lineage) extracted from EMSs, such as MLflow, with provenance information from VCSs, such as Git. That enables dependencies, interrelationships, and evolutions of ML experiment pipelines in collaborative environments to be captured, queried, and analyzed. We argue that MLflow2PROV's approach is complementary to record-level provenance tracking approaches (see also Table 3). In addition, MLflow2PROV's capturing approach is unique in the field of related work: by building on MLflow, many popular frameworks are supported, and no separate log statements are required.

7. Conclusion

In this paper, we present an approach for capturing end-to-end provenance of ML pipelines and their artifacts used and produced. We argue that the holistic provenance of ML pipelines requires considering any activities within associated ML AMS instances and VCS repositories. Based on MLflow as AMS and Git as VCS, we present a comprehensive provenance model compliant with the W3C PROV standard. Furthermore, we present our tool MLflow2PROV that continuously extracts provenance information from given MLflow and Git resources, generates provenance graphs according to our model, and enables querying, analyzing, and further processing of collected end-to-end provenance.

CRedit authorship contribution statement

Marius Schlegel: Writing – original draft, Visualization, Validation, Software, Resources, Methodology, Investigation, Formal analysis, Conceptualization. **Kai-Uwe Sattler:** Writing – review & editing, Supervision, Project administration, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was partially funded by the Thuringian Ministry of Economic Affairs, Science and Digital Society in the context of the project “Learning Products” (grant 5575/10-3).

Data availability

<https://github.com/mariusschlegel/mlflow2prov/>.

References

- [1] V. Chaoji, R. Rastogi, G. Roy, Machine Learning in the Real World, PVLDB 9 (13) (2016) 1597–1600.
- [2] S. Amershi, A. Begel, C. Bird, R. DeLine, H.C. Gall, E. Kamar, N. Nagappan, B. Nushi, T. Zimmermann, Software Engineering for Machine Learning: A Case Study, in: SEIP@ICSE '19, 2019, pp. 291–300.
- [3] M. Schlegel, K.-U. Sattler, Management of Machine Learning Lifecycle Artifacts: A Survey, ACM SIGMOD Rec. 51 (4) (2022) 18–35.
- [4] LF Projects, MLflow – A platform for the machine learning lifecycle, 2024, <https://mlflow.org>.
- [5] Weights & Biases, Weights & Biases – The AI Developer Platform, 2024, <https://wandb.ai>.
- [6] Allegro AI, ClearML – MLOps for Data Scientists, ML Engineers, and DevOps, 2024, <https://clear.ml>.
- [7] Polyaxon, Polyaxon – MLOps Tools For Managing & Orchestrating The Machine Learning Lifecycle, 2024, <https://polyaxon.com>.
- [8] Neptune.ai, Neptune.ai – ML Metadata Store, 2024, <https://neptune.ai>.
- [9] M. Zaharia, A. Chen, A. Davidson, A. Ghodsi, S.A. Hong, A. Konwinski, S. Murching, T. Nykodym, P. Ogilvie, M. Parkhe, F. Xie, C. Zumar, Accelerating the Machine Learning Lifecycle with MLflow, IEEE Data Eng. Bull. 41 (4) (2018) 39–45.
- [10] A. Chen, A. Chow, A. Davidson, A. DCunha, A. Ghodsi, S.A. Hong, A. Konwinski, C. Mewald, S. Murching, T. Nykodym, P. Ogilvie, M. Parkhe, A. Singh, F. Xie, M. Zaharia, R. Zang, J. Zheng, C. Zumar, Developments in MLflow: A System to Accelerate the Machine Learning Lifecycle, in: DEEM@SIGMOD '20, 2020, pp. 5:1–5:4.
- [11] P. Groth, L. Moreau, PROV-Overview: An Overview of the PROV Family of Documents, Tech. Rep., W3C, 2013.
- [12] P. Kairouz, H.B. McMahan, B. Avent, A. Bellet, M. Bennis, A.N. Bhagoji, K.A. Bonawitz, Z. Charles, G. Cormode, R. Cummings, R.G.L. D'Oliveira, H. Eichner, S.E. Rouayheb, D. Evans, J. Gardner, Z. Garrett, A. Gascón, B. Ghazi, P.B. Gibbons, M. Gruteser, Z. Harchaoui, C. He, L. He, Z. Huo, B. Hutchinson, J. Hsu, M. Jaggi, T. Javidi, G. Joshi, M. Khodak, J. Konečný, A. Korolova, F. Koushanfar, S. Koyejo, T. Lepoint, Y. Liu, P. Mittal, M. Mohri, R. Nock, A. Özgür, R. Pagh, H. Qi, D. Ramage, R. Raskar, M. Raykova, D. Song, W. Song, S.U. Stich, Z. Sun, A.T. Suresh, F. Tramèr, P. Vepakomma, J. Wang, L. Xiong, Z. Xu, Q. Yang, F.X. Yu, H. Yu, S. Zhao, Advances and Open Problems in Federated Learning, Found. Trends Mach. Learn. 14 (1–2) (2021) 1–210.
- [13] T.D. Huynh, L. Moreau, ProvStore: A Public Provenance Repository, in: IPAW '14, in: LNCS, vol. 8628, 2014, pp. 275–277.
- [14] Neo4j, Inc., Neo4j Graph Database & Analytics, 2024, <https://openprovenance.org/vis/>.
- [15] King's College London, ProvVis – Provenance Graph Visualisations, 2024, <https://openprovenance.org/vis/>.
- [16] D. Esteves, D. Moussallem, C.B. Neto, T. Soru, R. Usbeck, M. Ackermann, J. Lehmann, MEX Vocabulary: A Lightweight Interchange Format for Machine Learning Experiments, in: SEMANTICS '15, 2015, pp. 169–176.
- [17] G.C. Publio, D. Esteves, A. Lawrynowicz, P. Panov, L.N. Soldatova, T. Soru, J. Vanschoren, H. Zafar, ML-Schema: Exposing the Semantics of Machine Learning with Schemas and Ontologies, 2018, arXiv CoRR abs/1807.05351.

- [18] R. Souza, L.G. Azevedo, V. Lourenço, E.F.S. Soares, R. Thiago, R. Brandão, D. Civitarese, E.V. Brazil, M.F. Moreno, P. Valdúriez, M. Mattoso, R. Cerqueira, M.A.S. Netto, Workflow Provenance in the Lifecycle of Scientific Machine Learning, *Concurr. Comput. Pract. Exp.* 34 (14) (2022).
- [19] D.B. Pina, A. Chapman, L.N.O. Kunstmann, D. de Oliveira, M. Mattoso, DLProv: A Data-Centric Support for Deep Learning Workflow Analyses, in: DEEM@SIGMOD '24, ACM, 2024, pp. 77–85.
- [20] F. Psallidas, M.E. Leszczynski, M.H. Namaki, A. Floratou, A. Agrawal, K. Karanasos, S. Krishnan, P. Subotic, M. Weimer, Y. Wu, Y. Zhu, Demonstration of Geyser: Provenance Extraction and Applications over Data Science Scripts, in: SIGMOD '23, ACM, 2023, pp. 123–126.
- [21] M. Schlegel, K.-U. Sattler, MLflow2PROV: Extracting Provenance from Machine Learning Experiments, in: DEEM@SIGMOD '23, 2023, pp. 9:1–9:4.
- [22] M. Schlegel, K.-U. Sattler, Extracting Provenance of Machine Learning Experiment Pipeline Artifacts, in: ADBIS '23, in: LNCS, vol. 13985, 2023, pp. 238–251.
- [23] P. Cortez, A. Cerdeira, F. Almeida, T. Matos, J. Reis, Modeling wine preferences by data mining from physicochemical properties, *Decis. Support Syst.* 47 (4) (2009) 547–553.
- [24] P. Cortez, A. Cerdeira, F. Almeida, T. Matos, J. Reis, Wine Quality, 2009, <http://dx.doi.org/10.24432/C56S3T>, UCI Machine Learning Repository.
- [25] Taxi and Limousine Commission (TLC), TLC Trip Record Data, 2024, <https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page>.
- [26] L. Moreau, P. Missier, K. Belhajjame, R. B'Far, J. Cheney, S. Coppens, S. Cresswell, Y. Gil, P. Groth, G. Klyne, T. Lebo, J. McCusker, S. Miles, J. Myers, S. Sahoo, C. Tilmes, PROV-DM: The PROV Data Model, Tech. Rep., W3C, 2013, <https://www.w3.org/TR/prov-dm/>.
- [27] A. Schreiber, C. de Boer, L. von Kurnatowski, GitLab2PROV – Provenance of Software Projects hosted on GitLab, in: TaPP '21, 2021, pp. 1–9.
- [28] H. Percival, B. Gregory, Architecture Patterns with Python, O'Reilly, 2020.
- [29] R.C. Martin, Agile Software Development, Principles, Patterns, and Practices, Prentice Hall, ISBN: 978-0135974445, 2003.
- [30] S. Samuel, B. König-Ries, End-to-End provenance representation for the understandability and reproducibility of scientific experiments using a semantic approach, *J. Biomed. Semant.* 13 (1) (2022).
- [31] S. Soiland-Reyes, P. Sefton, M. Crosas, L.J. Castro, F. Coppens, J.M. Fernández, D. Garijo, B.A. Grüning, M. La Rosa, S. Leo, E.Ó. Carragáin, M. Portier, A. Trisovic, R. Community, P. Groth, C.A. Goble, Packaging research artefacts with RO-Crate, *Data Sci.* 5 (2) (2022) 97–138.
- [32] J. Vanschoren, J.N. van Rijn, B. Bischl, L. Torgo, OpenML: networked science in machine learning, *SIGKDD Explor.* 15 (2) (2013) 49–60.
- [33] University of Technology Sydney, The University of Manchester UK and RO-Crate contributors, RO-Crate Profiles, 2024, <https://www.researchobject.org/ro-crate/profiles.html>.
- [34] Isaac, Antoine and Summers, Ed, SKOS Simple Knowledge Organization System Primer, 2009, <https://www.w3.org/TR/skos-primer/>.
- [35] S. Leo, M.R. Crusoe, L. Rodríguez-Navas, R. Sirvent, A. Kanitz, P. De Geest, R. Wittner, L. Pireddu, D. Garijo, J.M. Fernández, I. Colonnelli, M. Gallo, T. Ohta, H. Suetake, S. Capella-Gutierrez, R. de Wit, B. de Paula Kinoshita, S. Soiland-Reyes, Recording provenance of workflow runs with RO-Crate, 2023, arXiv CoRR, [abs/2312.07852](https://arxiv.org/abs/2312.07852).
- [36] S. Leo, M.R. Crusoe, L. Rodríguez-Navas, R. Sirvent, A. Kanitz, P. De Geest, R. Wittner, L. Pireddu, D. Garijo, J.M. Fernández, I. Colonnelli, M. Gallo, T. Ohta, H. Suetake, S. Capella-Gutierrez, R. de Wit, B. de Paula Kinoshita, S. Soiland-Reyes, Recording provenance of workflow runs with RO-Crate (RO-Crate and mapping), 2023, <http://dx.doi.org/10.5281/zenodo.10368990>, Zenodo.
- [37] A. Chapman, L. Lauro, P. Missier, R. Torlone, DPDS: Assisting Data Science with Data Provenance, *PVLDB* 15 (12) (2022) 3614–3617.
- [38] S. Schelter, J.-H. Böse, J. Kirschnick, T. Klein, S. Seufert, Automatically Tracking Metadata and Provenance of Machine Learning Experiments, in: *MLSys@NIPS '17*, 2017, pp. 1–8.
- [39] S. Grafberger, S. Guha, J. Stoyanovich, S. Schelter, MLINSPECT: A Data Distribution Debugger for Machine Learning Pipelines, in: *SIGMOD '21*, 2021, pp. 2736–2739.
- [40] D. Kerzel, S. Samuel, B. König-Ries, Towards Tracking Provenance from Machine Learning Notebooks, in: *KDIR '21*, SCITEPRESS, 2021, pp. 274–281.
- [41] M.H. Namaki, A. Floratou, F. Psallidas, S. Krishnan, A. Agrawal, Y. Wu, Y. Zhu, M. Weimer, Vamsa: Automated Provenance Tracking in Data Science Scripts, in: *KDD '20*, 2020, pp. 1542–1551.